

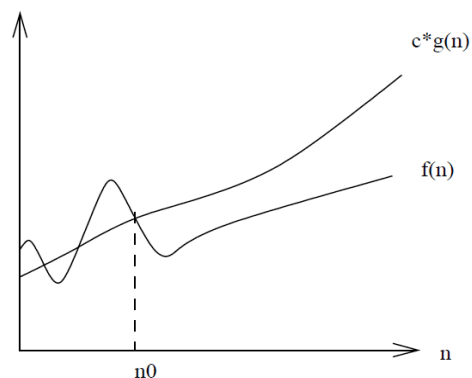
CSCI 423

Introduction to Algorithms

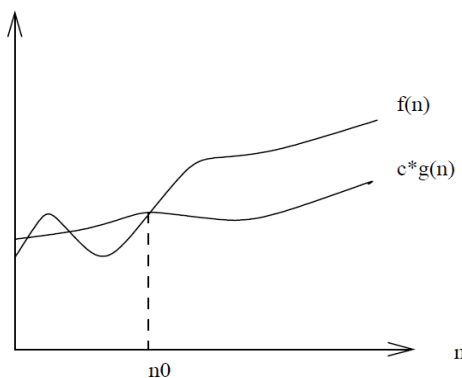


DR. RAAFAT ELFOULY

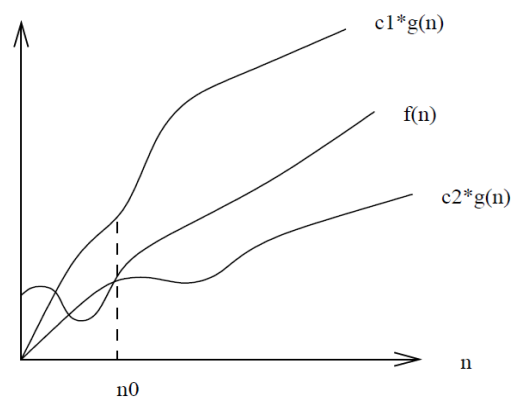
O , Ω , and Θ



(a)



(b)



(c)

The definitions imply a constant n_0 *beyond which* they are satisfied. We do not care about small values of n .

Useful summation formulas and rules

3

$$\sum_{l \leq i \leq u} 1 = 1 + 1 + \cdots + 1 = u - l + 1$$

In particular, $\sum_{1 \leq i \leq n} 1 = n - 1 + 1 = n \in \Theta(n)$

$$\sum_{1 \leq i \leq n} i = 1 + 2 + \cdots + n = n(n+1)/2 \approx n^2/2 \in \Theta(n^2)$$

$$\sum_{1 \leq i \leq n} i^2 = 1^2 + 2^2 + \cdots + n^2 = n(n+1)(2n+1)/6 \approx n^3/3 \in \Theta(n^3)$$

$$\sum_{1 \leq i \leq n} i^3 = 1^3 + 2^3 + \cdots + n^3 = n^2(n+1)(n+1)/4 \approx n^4/4 \in \Theta(n^4)$$

$$\sum_{0 \leq i \leq n} a^i = 1 + a + \cdots + a^n = (a^{n+1} - 1)/(a - 1) \text{ for any } a \neq 1$$

In particular, $\sum_{0 \leq i \leq n} 2^i = 2^0 + 2^1 + \cdots + 2^n = 2^{n+1} - 1 \in \Theta(2^n)$

$$\sum (a_i \pm b_i) = \sum a_i \pm \sum b_i \quad \sum c a_i = c \sum a_i \quad \sum_{l \leq i \leq u} a_i = \sum_{l \leq i \leq m} a_i + \sum_{m+1 \leq i \leq u} a_i$$

Advanced Dominance Rankings

Additional functions arise in more sophisticated analysis than we will do in this course:

$$n! \gg c^n \gg n^3 \gg n^2 \gg n^{1+\epsilon} \gg n \log n \gg n \gg \sqrt{n} \gg \log^2 n \gg \log n \gg \log n / \log \log n \gg \log \log n \gg \alpha(n) \gg 1$$

Approximating Recurrence Relations

5

The Master Method

If $T(n) \leq aT\left(\frac{n}{b}\right) + O(n^d)$

then

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \quad (\text{Case 1}) \\ O(n^d) & \text{if } a < b^d \quad (\text{Case 2}) \\ O(n^{\log_b a}) & \text{if } a > b^d \quad (\text{Case 3}) \end{cases}$$

Chapter 3



SEARCHING AND SELECTION ALGORITHMS

Chapter Outline

7

- Sequential search
- Binary search
- List Selection

Prerequisites

8

- Before beginning this chapter, you should be able to:
 - Read and create algorithms
 - Use mathematical concepts presented in Chapter 1,2

Goals

9

- At the end of this chapter you should be able to:
 - Explain the sequential search algorithm and its worst-case and average-case analysis
 - Explain the binary search algorithm and its worst-case and average-case analysis
 - Explain the selection algorithms and their analysis

Sequential Search

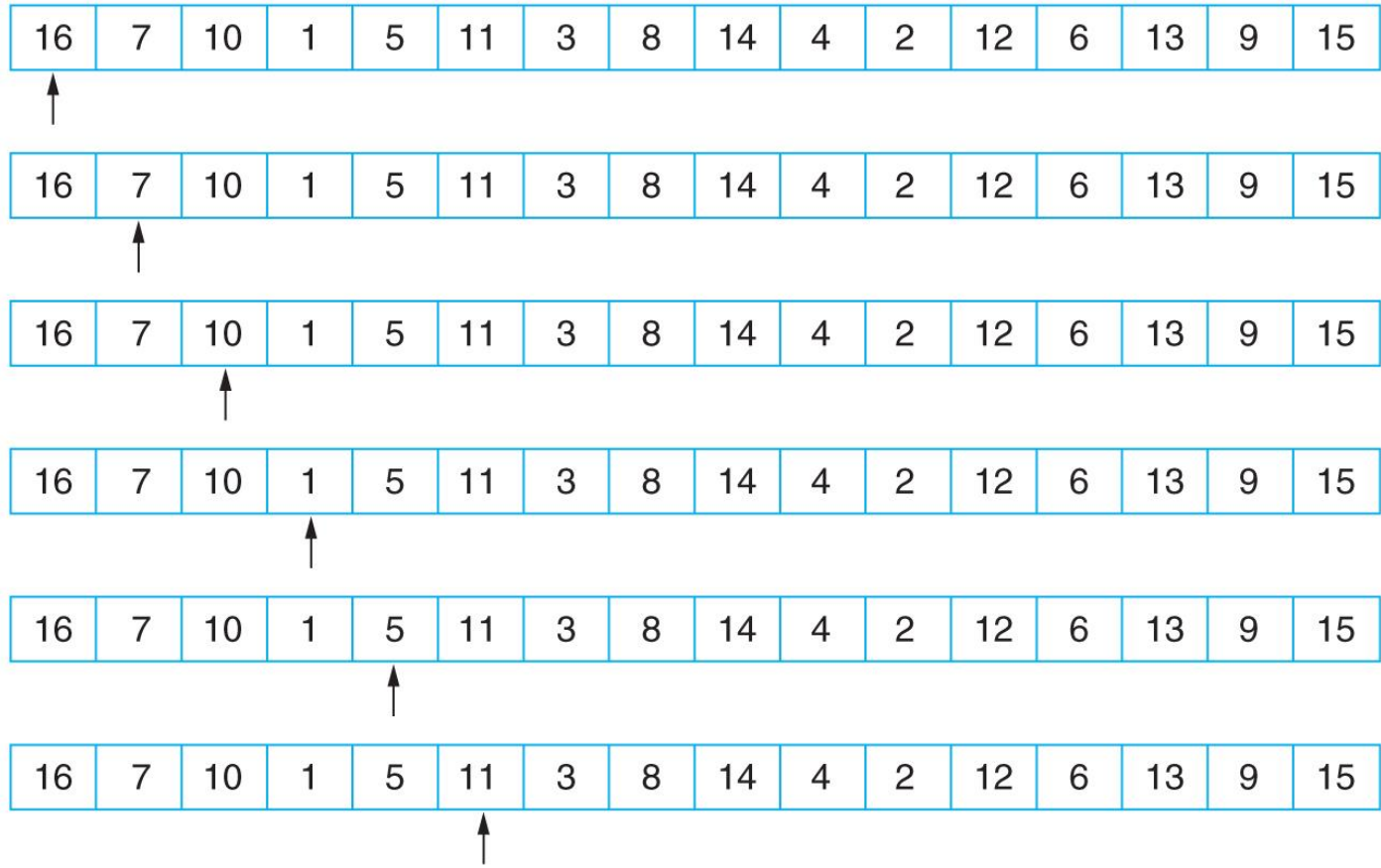
10

- Looks for the target from the first to the last element of the list
- The later in the list the target occurs the longer it takes to find it
- Does not assume anything about the order of the elements in the list, so it can be used with an unsorted list

Sequential Search Example

11

■ **FIGURE 3.1**
Sequential search



Sequential Search Algorithm

12

```
for i = 1 to N do
    if (target == list[i])
        return I
    Break
end for
return 0
```

Worst-Case Analysis

13

- If the target is in the last location, we look at all of the elements to find it
- If the target is not in the list, we need to look at all of the elements to learn that
- Therefore, the largest number of comparisons we will do in this algorithm is N

Average-Case Analysis

14

- If the search is always successful, there are N places the target could be found
- It will take 1 comparison to find the target in the first location, 2 comparisons to find the target in the second location, and so on
- If each location is equally likely,
what shall we get?

Average-Case Analysis

15

- If the search is always successful, there are N places the target could be found
- It will take 1 comparison to find the target in the first location, 2 comparisons to find the target in the second location, and so on
- If each location is equally likely, we get:

$$A(N) = \frac{1}{N} \sum_{i=1}^N i = \frac{N+1}{2}$$

Average-Case Analysis

16

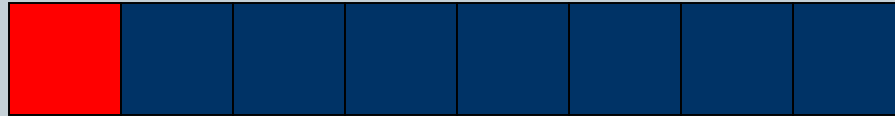
- If the search can fail, there are N places the target could be found and 1 possibility when it's not found
- If the target is not found, we do N comparisons
- If each of these $N+1$ possibilities are equally likely, we get:

$$A(N) = \frac{1}{N+1} \left(N + \sum_{i=1}^N i \right) \approx \frac{N+2}{2}$$

Recursive - Linear Search



- Recursively
- Look at an element (constant work, c), then search the remaining elements...



- $T(n) = T(n-1) + c$
- “The cost of searching n elements is the cost of looking at 1 element, plus the cost of searching $n-1$ elements”

Linear Search (cont.)



- We'll “unwind” a few of these

$$T(n) = T(n-1) + c \quad (1)$$

But, $T(n-1) = T(n-2) + c$, from above

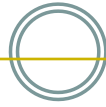
Substituting back in:

$$T(n) = T(n-2) + c + c$$

Gathering like terms

$$T(n) = T(n-2) + 2c \quad (2)$$

EX. 1 – list of intermediates



Result at i^{th} unwinding	i
$T(n) = T(n-1) + 1c$	1
$T(n) = T(n-2) + 2c$	2
$T(n) = T(n-3) + 3c$	3
$T(n) = T(n-4) + 4c$	4

Linear Search (cont.)



- An expression for the kth unwinding:
$$T(n) = T(n-k) + kc$$
- We have 2 variables, k and n, but we have a relation
- $T(d)$ is constant (can be determined) for some constant d (we know the algorithm)
- Choose any convenient # to stop.

Linear Search (cont.)



- Let's decide to stop at $T(o)$. When the list to search is empty, you're done...
- o is convenient, in this example...

$$\text{Let } n-k = o \Rightarrow n=k$$

- Now, substitute n in everywhere for k :

$$T(n) = T(n-n) + nc$$

$$T(n) = T(o) + nc = nc + c_o = O(n)$$

($T(o)$ is some constant, c_o)

Binary Search

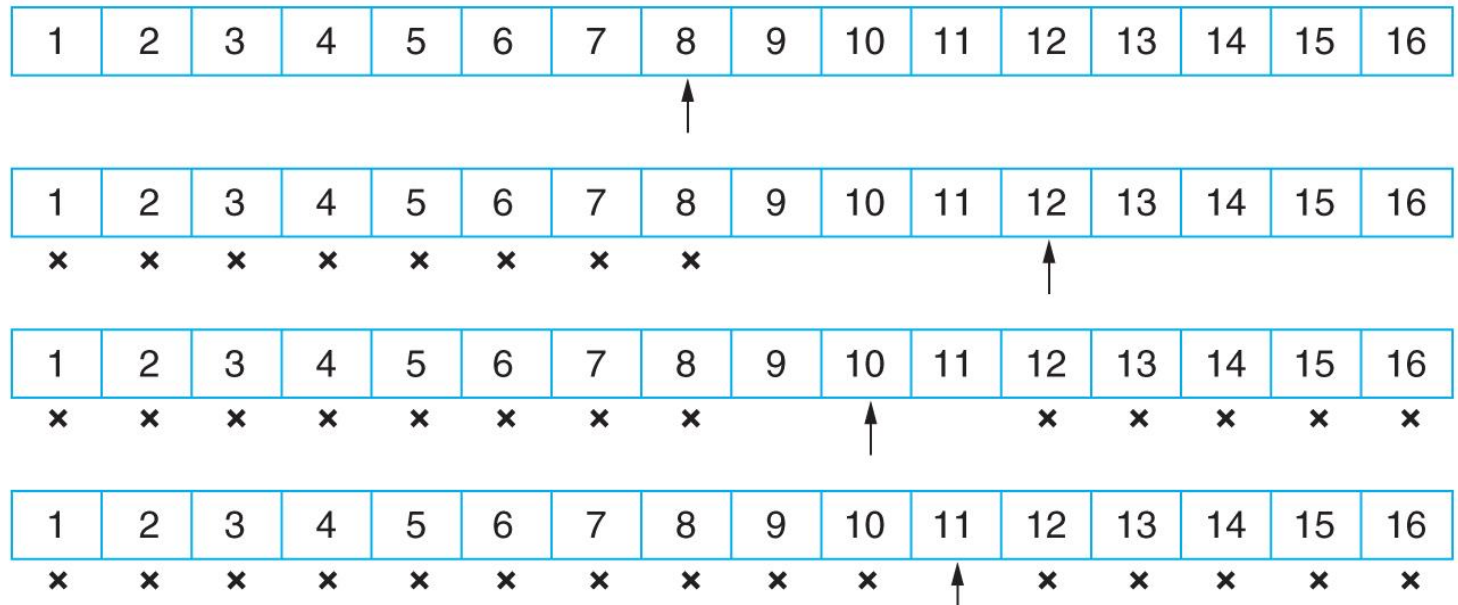
22

- Used with a sorted list
- First check the middle list element
- If the target matches the middle element, we are done
- If the target is less than the middle element, the key must be in the first half
- If the target is larger than the middle element, the key must be in the second half

Binary Search Example

23

FIGURE 3.2
Binary search



Binary Search Algorithm

24

```
start = 1
end = N
while start ≤ end do
  middle = (start + end) / 2
  switch (Compare(list[middle], target))
    case -1:  start = middle + 1
              break
    case  0:  return middle
              break
    case  1:  end = middle - 1
              break
  end select
end while
return 0
```


Algorithm Review

25

- Each comparison eliminates about half of the elements of the list from consideration
- If we begin with $N = 2^k - 1$ elements in the list, there will be $2^{k-1} - 1$ elements on the second pass, and $2^{k-2} - 1$ elements on the third pass

Worst-Case Analysis

26

- In the worst case, we will either find the target on the last pass, or not find the target at all
- The last pass will have only one element left to compare, which happens when $2^1 - 1 = 1$
- If $N = 2^k - 1$, then there must be $k = \lg(N+1)$ passes

Average-Case Analysis

27

- If the search is always successful, there are N places the target could be found
- There is one place we check on the first pass, two places we could check on the second pass, and four places we could check on the third pass

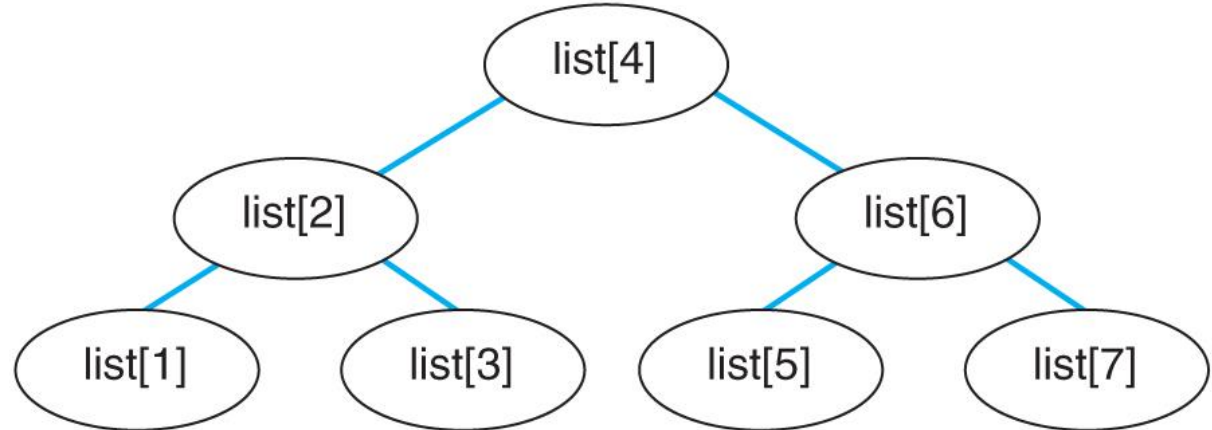
Average-Case Analysis

28

- We can represent binary search as a binary tree:

■ **FIGURE 3.3**

Decision tree for a search of a list of seven elements



Average-Case Analysis

29

- In looking at the binary tree, we see that there are i comparisons needed to find the 2^{i-1} elements on level i of the tree
- For a list with $N = 2^k - 1$ elements, there are k levels in the binary tree
- These two facts give us:

$$A(N) = \frac{1}{N} \sum_{i=1}^k i * 2^{i-1} \approx \lg(N+1) - 1$$

Binary Search – Recurrence relation



- Algorithm – “check middle, then search lower $\frac{1}{2}$ or upper $\frac{1}{2}$ ”
- $T(n) = T(n/2) + c$

where c is some constant, the cost of checking the middle...

Binary Search (cont)



Let's do some quick substitutions:

$$T(n) = T(n/2) + c \quad (1)$$

but $T(n/2) = T(n/4) + c$, so

$$T(n) = T(n/4) + c + c$$

$$T(n) = T(n/4) + 2c \quad (2)$$

$$T(n/4) = T(n/8) + c$$

$$T(n) = T(n/8) + c + 2c$$

$$T(n) = T(n/8) + 3c \quad (3)$$

Binary Search (cont)



Result at i^{th} unwinding

i

$$T(n) = T(n/2) + c$$

$$= T(n/2^1) + 1c$$

1

$$T(n) = T(n/4) + 2c$$

$$= T(n/2^2) + 2c$$

2

$$T(n) = T(n/8) + 3c$$

$$= T(n/2^3) + 3c$$

3

$$T(n) = T(n/16) + 4c$$

$$= T(n/2^4) + 4c$$

4

Binary Search (cont)



- After k unwindings:

$$T(n) = T(n/2^k) + kc$$

- Need a convenient place to stop unwinding – need to relate k & n

- Let's consider $T(1) = c_0$

- So, let:

$$n/2^k = 1 \quad \Rightarrow$$

$$n = 2^k \quad \Rightarrow$$

$$k = \log_2 n = \lg n$$

Binary Search (cont.)



- Substituting back in (getting rid of k):

$$T(n) = T(1) + c \lg(n)$$

$$= c \lg(n) + c_0$$

$$= O(\lg(n))$$

Approximating Recurrence Relations

35

The Master Method

If $T(n) \leq aT\left(\frac{n}{b}\right) + O(n^d)$

then

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \quad (\text{Case 1}) \\ O(n^d) & \text{if } a < b^d \quad (\text{Case 2}) \\ O(n^{\log_b a}) & \text{if } a > b^d \quad (\text{Case 3}) \end{cases}$$

Searching with presorting

36

Problem: Search for a given K in $A[0..n-1]$

Presorting-based algorithm:

Stage 1 Sort the array by an efficient sorting algorithm

Stage 2 Apply binary search

Efficiency: $\Theta(n \log n) + O(\log n) = \Theta(n \log n)$

Good or bad?

Why do we have our dictionaries, telephone directories, etc. sorted?

Selection

37

- We want to find a particular element of an unsorted list
- Typically, this is expressed as wanting to find the K^{th} largest element
- We could find this by first sorting the entire list, but we will see that is more work than we need to do ($O(n \log n)$) so we hope to find better algorithm!!

First Selection Algorithm

38

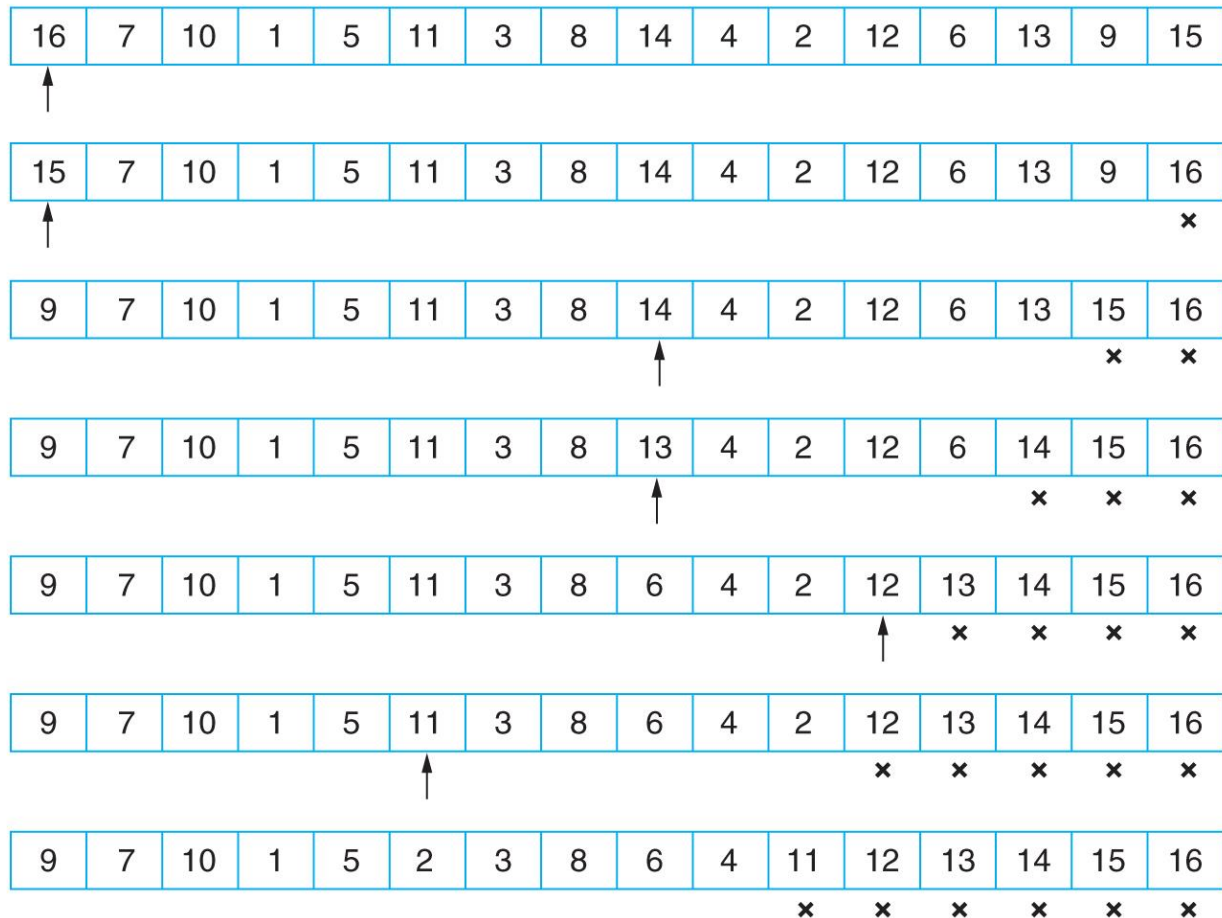
- To find the K^{th} largest element, we could move the K largest elements to the end of the list and then the K^{th} largest will be in location $\text{list}[K]$
- This involves finding the largest element and moving it to the end, then finding the second largest element and moving it to the end and so on

First Selection Algorithm Example

39

■ **FIGURE 3.4**

Finding the K^{th} largest value by moving large values to the end of the list



First Selection Algorithm

40

```
For i = 1 to K do
    largest = list[1]
    largestLocation = 1
    for j = 2 to N-(i-1) do
        if list[j] > largest then
            largest = list[j]
            largestLocation = j
        end if
    end for
    Swap( list[N-(i-1)], list[largestLocation] )
end for
return largest
```


First Selection Algorithm

41

```
For i = 1 to K do
    largest = list[1]
    largestLocation = 1
    for j = 2 to N-(i-1) do
        if list[j] > largest then
            largest = list[j]
            largestLocation = j
        end if
    end for
    Swap( list[N-(i-1)], list[largestLocation] )
end for
return largest
```

Complexity???

First Algorithm Analysis

42

- On the first pass, we compare the first element to the rest, doing $N - 1$ comparisons
- On the second pass, we compare the first element to the rest, doing $N - 2$ comparisons
- We need to do K passes, giving

$$\sum_{i=1}^K N - i = N * K - \frac{K * (K + 1)}{2} = O(N * K)$$

Few Announcements



- Mid term Oct 9th

HW3

Due Oct 16th

44

- Write a program to implement one of the following algorithms
 - Selection search
 - Binary search
- Run the program for different input sizes (10K, 1Million, and 100 Millions)
- Measure the execution time using profile function and compare it to each algorithm complexity.
- Draw the execution time as function of input sample

Project 1 Search Algorithms Survey

Due Oct 28th

45

- You are required to do research on the state of the art/most recent search algorithms/Search Engines used in one of the following applications: Google, Facebook, Amazon ...)
- Steps:
 - Choose two recent papers (2017+) describing search algorithm/Engines in one of the above applications
 - Summarize your findings and compare complexity of the new search to traditional ones (linear and binary)
 - Write a program to implement one of the algorithms
- Deliverables:
 - Summary document describing your findings (3-4 pages at most). Including complexity of the new algorithms and graph it.
 - Program showing your implementation.

Second Selection Algorithm

46

- The first algorithm does more work than necessary, because we do not need to know the order of the larger elements, just that they are larger
- We now pick an element and partition the list into two parts with those larger toward the front and those smaller toward the end

Selection Problem

47

Find the k -th smallest element in a list of n numbers

- $k = 1$ or $k = n$

- median: $k = \lceil n/2 \rceil$

Example: 4, 1, 10, 9, 7, 12, 8, 2, 15 median = ?

The median is used in statistics as a measure of an average value of a sample. In fact, it is a better (more robust) indicator than the mean, which is used for the same purpose.

Selection Problem

48

Find the k -th smallest element in a list of n numbers

- $k = 1$ or $k = n$
- median: $k = \lceil n/2 \rceil$

Example: 4, 1, 10, 9, 7, 12, 8, 2, 15 median = 8

The **median** is also the number that is halfway into the set.

To find the **median**, the data should be arranged in order from least to greatest. If there is an even number of items in the data set, then the **median** is found by taking the mean (average) of the two middlemost numbers.

Algorithms for the Selection Problem

49

The sorting-based algorithm: Sort and return the k -th element

Efficiency (if sorted by mergesort): $\Theta(n \log n)$

A faster algorithm is based on the array partitioning:

all are $\leq A[s]$		all are $\geq A[s]$
---------------------	--	---------------------

Assuming that the array is indexed from 0 to $n-1$ and s is a split position obtained by the array partitioning:

If $s = k-1$, the problem is solved;

if $s > k-1$, look for the k -th smallest element in the left part;

if $s < k-1$, look for the $(k-s)$ -th smallest element in the right part.

Note: The algorithm can simply continue until $s = k-1$.

Second Selection Algorithm

50

- If the partitioning element winds up at location $P = K$, we have found the K^{th} largest element
- If the partitioning element winds up at location $P > K$, the element we want is in the first part
- If the partitioning element winds up at location $P < K$, the element we want is in the second part

51

Diagram illustrating the step-by-step construction of a binary search tree from an array. The array is [16, 7, 10, 1, 5, 11, 3, 8, 14, 4, 2, 12, 6, 13, 9, 15]. The root node is 16. Its left child is 7, and its right child is 10. Node 7 has a left child 1, and node 10 has a right child 11. Node 1 has a right child 5, and node 11 has a left child 3. Node 5 has a right child 8, and node 3 has a right child 14. Node 8 has a left child 4, and node 4 has a left child 2. Node 2 has a right child 12, and node 12 has a left child 6. Node 6 has a right child 13, and node 13 has a left child 9. Node 9 has a right child 15.

Second Selection Algorithm

52

```
if start < end then
  Partition( list, start, end, middle )
  if middle == K then
    return list[middle]
  else
    if K < middle then
      return KthLargestRecursive(list, start,
                                middle-1, K)
    else
      return KthLargestRecursive(list, middle+1,
                                end, K-(middle-start+1))
    end if
  end if
end if
```

Second Algorithm Analysis

53

- The partition function will do about N comparisons for a list of size N
- A simple analysis assumes that the list is divided in half each time
- This gives the comparison equation

$$N + \frac{N}{2} + \frac{N}{4} + \frac{N}{8} + \cdots + 1 \approx 2N$$

Approximating Recurrence Relations

54

The Master Method

If $T(n) \leq aT\left(\frac{n}{b}\right) + O(n^d)$

then

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \quad (\text{Case 1}) \\ O(n^d) & \text{if } a < b^d \quad (\text{Case 2}) \\ O(n^{\log_b a}) & \text{if } a > b^d \quad (\text{Case 3}) \end{cases}$$



- Always $O(N)$